

PRACTICAL 12

Q1. Producer-Consumer Using Semaphores

Aim

To implement the Producer-Consumer problem using counting semaphores and POSIX threads.

Theory

The Producer-Consumer problem involves a producer adding items to a shared buffer and a consumer removing them.

Semaphores are used to control access to the buffer.

Program

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;

void *producer(void *arg) {
    for (int i = 1; i <= 10; i++) {
        sem_wait(&empty);

        buffer[in] = i;
        printf("Produced: %d\n", i);

        in = (in + 1) % SIZE;

        sem_post(&full);
    }

    return NULL;
}
```

```

void *consumer(void *arg) {
    for (int i = 1; i <= 10; i++) {

        sem_wait(&full);

        printf("Consumed: %d\n", buffer[out]);

        out = (out + 1) % SIZE;

        sem_post(&empty);
    }

    return NULL;
}

int main() {
    pthread_t p, c;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);

    return 0;
}

```

Output

```

amrutha@blue:~$ nano producer.c
amrutha@blue:~$ gcc producer.c -o producer
amrutha@blue:~$ ./producer
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9
Produced: 10
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10

```

Observation

The producer adds items to the buffer and the consumer removes them. Semaphores prevent the producer from filling a full buffer and the consumer from accessing an empty buffer.

Result

Thus, the Producer-Consumer problem was successfully implemented using POSIX threads and counting semaphores.

Q2. Deadlock and Deadlock Prevention Using Resource Ordering

Aim

To demonstrate a deadlock scenario involving multiple threads and shared resources and implement a deadlock prevention strategy using resource ordering.

Theory

Deadlock occurs when two or more threads wait for resources held by each other. The four necessary conditions for deadlock are:

Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait

Resource ordering prevents circular wait by making all threads acquire resources in the same order.

Program

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t r1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t r2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg) {
    pthread_mutex_lock(&r1);
    printf("Thread 1 acquired Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&r2);
    printf("Thread 1 acquired Resource 2\n");

    pthread_mutex_unlock(&r2);
    pthread_mutex_unlock(&r1);

    return NULL;
}
```

```

void *thread2(void *arg) {
    pthread_mutex_lock(&r2);
    printf("Thread 2 acquired Resource 2\n");

    sleep(1);

    pthread_mutex_lock(&r1);
    printf("Thread 2 acquired Resource 1\n");

    pthread_mutex_unlock(&r1);
    pthread_mutex_unlock(&r2);

    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}

```

Execution and Output

```

amrutha@blue:~$ nano deadlock.c
amrutha@blue:~$ gcc deadlock.c -o deadlock
amrutha@blue:~$ ./deadlock
Thread 1 acquired Resource 1
Thread 2 acquired Resource 2

```

gcc deadlock.c -o deadlock -pthread

./deadlock

The program may stop responding because each thread is waiting for a resource held by the other.

Deadlock Prevention

Both threads should acquire resources in the same order:

Resource 1 → Resource 2

For example:

```
pthread_mutex_lock(&r1);
```

```
pthread_mutex_lock(&r2);
```

```
/* Critical section */
```

```
pthread_mutex_unlock(&r2);
```

```
pthread_mutex_unlock(&r1);
```

If every thread follows the same resource order, circular waiting is avoided.

Observation

The original program can cause deadlock because the threads acquire the resources in different orders.

Result

Thus, a deadlock scenario was demonstrated and deadlock prevention using resource ordering was implemented.